Probability Theory and Language Model - Code Explanation

This program demonstrates Probability Theory in NLP using Unigram, Bigram, and Trigram Language Models. It calculates the probability of the sentence: I love machine learning.

Step 1: Import Counter from collections to count word frequencies automatically.

Step 2: Create a sample corpus containing five sentences related to machine learning and AI.

Step 3: Convert each sentence to lowercase and split it into words. This process is called tokenization.

Step 4: Display the tokenized corpus.

Step 5: Count individual words using a Unigram model. Counter stores the frequency of each word. Total words are calculated using sum(unigram_counts.values()).

Step 6: Count Bigrams (pairs of consecutive words). Examples: (i,love), (love,machine), (machine,learning).

Step 7: Count Trigrams (groups of three consecutive words). Examples: (i,love,machine) and (love,machine,learning).

Step 8: Define the test sentence 'I love machine learning' and tokenize it.

Step 9: Compute Unigram Probability using the formula P(w1) x P(w2) x P(w3) x P(w4).

Step 10: Compute Bigram Probability using conditional probability P(wi|wi-1).

Step 11: Compute Trigram Probability using conditional probability P(wi|wi-2,wi-1).

Step 12: Display word counts, bigram counts, trigram counts and sentence probabilities.

NLP Concept:
Unigram considers each word independently.
Bigram considers the previous word.
Trigram considers the previous two words.
As we move from Unigram to Bigram to Trigram, more context is used and sentence prediction becomes more accurate.